LESK ALGORITHM — COMPLETE TEACHING & REFERENCE GUIDE

Word Sense Disambiguation (WSD): theory, algorithm, worked sums, variants, limitations, and teaching notes

Prepared from the four uploaded course resources (WSD notes, Lesk Algorithm notes, Module lecture slides, and WSD notes) and cross-checked against the original Lesk publication, official WordNet documentation, NLTK documentation, and later Lesk extensions.

Learning objectives: Explain WSD contextually; distinguish original and simplified Lesk; calculate overlap scores; solve tie cases; explain Corpus/IDF and extended Lesk; and identify limitations.

1. What students should already know

The students already have the basic idea of Word Sense Disambiguation (WSD): the same word can have different meanings, and WSD selects the meaning appropriate to a particular context. For example, “bass” can refer to a low musical frequency or a fish, depending on the sentence. The uploaded course material places Lesk under the knowledge-based / dictionary-based family of WSD methods.

The Lesk algorithm does not learn a classifier from a large labelled training set. Instead, it uses lexical knowledge: dictionary definitions (glosses), example sentences, and, in extended versions, lexical relations such as those represented in WordNet.

2. Lesk in one sentence

Lesk chooses the sense of an ambiguous target word whose dictionary description has the greatest useful word overlap with the surrounding context.

The central intuition is: words used together in a sentence tend to be semantically related, and that relatedness can be detected by overlap between their dictionary definitions.

3. Important terminology

Target word: the ambiguous word whose meaning we want to determine.

Sense: one particular meaning of a word. A word such as “bank” may have a financial sense and a river-side sense.

Gloss: the dictionary definition of a sense. In WordNet, a synset has a gloss containing a definition and optionally example sentences.

Synset: a WordNet set of roughly synonymous words/collocations representing a particular concept. A word form can occur in multiple synsets, corresponding to different senses.

Context: the words surrounding the target word. Depending on the version of Lesk, the context may be the whole sentence, a local window, or the definitions/signatures of nearby words.

Signature: the textual representation used for a sense, normally its gloss plus example sentences; Corpus Lesk can add sense-labelled corpus material.

Overlap: the words or phrases shared by two pieces of text after the chosen preprocessing.

Stop words / function words: very common grammatical words such as “the”, “a”, “and”, “of”. Basic Lesk implementations commonly ignore them because they provide little semantic evidence.

Context window: the selected set of nearby words used as evidence. A larger window gives more evidence but can also introduce irrelevant words.

4. Why Lesk is a knowledge-based WSD method

WSD methods can be grouped broadly into supervised, dictionary/knowledge-based, semi-supervised, and unsupervised approaches. The course material explicitly lists dictionary/thesaurus methods and identifies Lesk as the seminal dictionary-based method.

Lesk relies on a pre-existing lexical resource rather than requiring a sense-labelled training corpus. WordNet is a common resource because it provides synsets, glosses, examples, and semantic relations. WordNet's official documentation states that each synset contains a gloss consisting of a definition and optionally example sentences.

This is an important distinction for teaching:
• Supervised WSD learns from labelled examples.
• Basic Lesk reasons from dictionary knowledge.
• Corpus Lesk is a hybrid extension because it enriches the sense signature with sense-labelled corpus evidence.

5. The basic mathematical idea

For a target word w with candidate senses s1, s2, …, sk, let Signature(si) be the words in the gloss/examples of sense si and Context be the processed context.

For the simplest overlap score:

```
Score(si, Context) = | Signature(si) ∩ Context |
```

The selected sense is:

```
s* = argmax_si Score(si, Context)
```

In plain language: calculate the number of useful words shared by each candidate sense and the context, then select the sense with the largest score.

Important: ordinary simplified Lesk is a lexical-overlap method. It does not automatically understand that “car” and “automobile” are semantically related unless the representation or resource makes that relationship available. Exact or normalized lexical overlap is the basic signal.

6. Original Lesk algorithm — what Lesk originally proposed

The original Lesk formulation is more indirect than the commonly taught simplified Lesk algorithm.

Instead of directly comparing the target sense's gloss with the raw context words, original Lesk compares the gloss/signature of the target sense with the glosses/signatures of the senses of surrounding words. The sense combination with the greatest total overlap provides the evidence for the interpretation.

The classic example in the course material is “pine cone”:

Pine:
• pine1: “kinds of evergreen tree with needle-shaped leaves”
• pine2: “waste away through sorrow or illness”

Cone:
• cone1: “solid body which narrows to a point”
• cone2: “something of this shape whether solid or hollow”
• cone3: “fruit of certain evergreen trees”

For cone3, the words “evergreen” and “tree” overlap with the definition of pine1. The other cone senses do not obtain that overlap. Therefore cone3 is selected.

This example is important because it demonstrates the original Lesk idea: the surrounding word “pine” supplies semantic evidence through its own dictionary definition.

7. Simplified Lesk — the version most useful for classroom calculations

Simplified Lesk changes the comparison to something easier:

1. Select the target word.
2. Collect its candidate senses.
3. Take the sentence/context containing the target.
4. Preprocess the context: tokenize, normalize case, remove punctuation, and usually remove stop words; lemmatization may also be used.
5. For each candidate sense, construct its signature from the gloss and, when available, example sentences.
6. Remove or normalize stop words in the signature in the same way.
7. Compute overlap between the signature and the context.
8. Record the score.
9. Select the sense with the maximum score.
10. If there is a tie, apply a stated tie-breaking rule rather than silently choosing a sense.

A standard simplified-Lesk pseudocode is:

```
SIMPLIFIED-LESK(word, context)
best_sense = default / most-frequent sense
max_score = 0

for each sense of word:
signature = gloss(sense) + examples(sense)
score = overlap(signature, context)
```

```
if score > max_score:
max_score = score
best_sense = sense

return best_sense
```

The uploaded course notes specifically describe the simplified algorithm as comparing a sense signature with the context and counting common words while ignoring function words / stop-list items.

8. What exactly counts as an overlap?

For an exam-style problem, use the rules stated in the question. If no rules are given, use a consistent preprocessing pipeline.

A common manual procedure is:

Raw text → lowercase → remove punctuation → tokenize → remove stop words → optionally lemmatize → compare sets.

Example:
“The bank accepts deposits and gives loans.”

After lowercasing and removing punctuation:
the, bank, accepts, deposits, and, gives, loans

After stop-word removal:
bank, accepts, deposits, gives, loans

If the target word itself is included in the context, it is normally not useful evidence for distinguishing its own senses, so exclude the target from the context set when doing a manual calculation.

If lemmatization is used:
deposits → deposit
gives → give
loans → loan

Do not mix preprocessing rules between the context and signatures. If you lemmatize one side, lemmatize the other side too.

For the basic set-overlap version, repeated occurrences do not automatically give multiple points. Convert to sets unless the problem explicitly defines frequency-weighted overlap.

9. Worked Example 1 — very basic numerical sum

TARGET WORD: bank

Sentence:
“I went to the bank to deposit money.”

Assume two candidate senses:

Sense 1 — financial bank:
“a financial institution that accepts deposits and handles money”

Sense 2 — river bank:
“the sloping land beside a river or other body of water”

Step 1: Process the context.

Raw context:
I, went, to, the, bank, to, deposit, money

Remove stop words and the target:
{went, bank, deposit, money} → {went, deposit, money}

Step 2: Convert each gloss to useful content words.

Signature 1:
{financial, institution, accepts, deposits, handles, money}

Signature 2:
{sloping, land, river, body, water}

Step 3: Calculate overlap.

```
Score(bank1) = |{financial, institution, accepts, deposits, handles, money}
∩ {went, deposit, money}|
= 1

Score(bank2) = |{sloping, land, river, body, water}
∩ {went, deposit, money}|
= 0
```

Therefore:
bank1 score = 1
bank2 score = 0

FINAL ANSWER:
The target word “bank” is assigned the financial-institution sense.

Teaching point:
Do not say “deposit” and “deposits” match unless you have explicitly allowed lemmatization/stemming. If lemmatization is applied, deposit ↔ deposits becomes an overlap and the score becomes 2 for bank1.

10. Worked Example 2 — advanced sum with preprocessing and IDF weighting

This example has two layers. First solve ordinary simplified Lesk. Then show how Corpus Lesk / IDF weighting changes the score.

TARGET WORD: plant

Sentence:
“The workers inspected the plant machines after production stopped.”

Candidate senses:

S1 — living plant:
“a living organism that grows in soil and produces leaves”

S2 — industrial plant:
“an industrial building where machines produce goods”

S3 — planting action:
“to put a seed in soil so that it can grow”

PART A — Ordinary simplified Lesk

After lowercasing, punctuation removal, stop-word removal, and lemmatization:

Context =
{worker, inspect, plant, machine, production, stop}

Remove the target “plant”:
C = {worker, inspect, machine, production, stop}

Use the following normalized signatures:

S1 = {live, organism, grow, soil, produce, leaf}
S2 = {industrial, building, machine, produce, good, production}
S3 = {put, seed, soil, grow}

Scores:

```
S1:
C ∩ S1 = ∅
Score(S1) = 0

S2:
C ∩ S2 = {machine, production}
Score(S2) = 2

S3:
C ∩ S3 = ∅
Score(S3) = 0
```

Therefore:
S1 = 0
S2 = 2
S3 = 0

FINAL SENSE = S2, the industrial/factory meaning.

PART B — IDF-weighted Corpus Lesk idea

Corpus Lesk gives more weight to informative overlapping words. A standard IDF form is:

IDF(w) = log(N / df(w))

and:

```
Score(s, C) = Σ IDF(w), for w in Signature(s) ∩ C
```

Suppose N = 1000 documents and the relevant document frequencies are:

machine: df = 50
production: df = 100

Then:

IDF(machine) = log(1000/50) = log(20) ≈ 2.996
IDF(production) = log(1000/100) = log(10) ≈ 2.303

Therefore:

Score_IDF(S2)
= 2.996 + 2.303
= 5.299

The other two senses have no overlap, so:

Score_IDF(S1) = 0
Score_IDF(S2) ≈ 5.299
Score_IDF(S3) = 0

The selected sense remains S2, but the evidence is now weighted: a rarer, more informative word contributes more than a very common word.

Teaching point:
Do not use IDF weighting in an ordinary Lesk sum unless the question explicitly gives document frequencies or asks for Corpus/IDF Lesk.

11. The textbook-style “bank” example

The uploaded WSD notes give a fuller simplified-Lesk example:

“The bank can guarantee deposits will eventually cover future tuition costs because it invests in adjustable-rate mortgage securities.”

The two candidate senses are approximately:

bank1: a financial institution that accepts deposits and channels money into lending activities; examples include cashing a check and holding a mortgage.

bank2: sloping land, especially the slope beside a body of water; examples include pulling a canoe onto the bank or sitting beside a river.

The course material reports that bank1 has two non-stopword overlaps with the context — “deposits” and “mortgage” — while bank2 has zero. Therefore bank1 is selected.

This is a particularly good classroom example because students can see the decision directly from the semantic clues in the sentence.

12. Tie cases — a point students often miss

Suppose two candidate senses both obtain the same maximum score.

Example:
S1 score = 2
S2 score = 2
S3 score = 0

The overlap calculation alone does not distinguish S1 and S2.

Possible tie-breaking strategies include:
• choose the most frequent sense, if frequency information is available;
• use a larger or better-defined context window;
• include example sentences in the signatures;
• use a weighted/extended Lesk variant;
• use another WSD method.

The important exam rule is: never invent a tie-breaker without stating it. If the question gives a tie-breaking rule, follow that rule.

13. Corpus Lesk

Corpus Lesk keeps the basic gloss-overlap intuition but enriches the signature using sense-labelled corpus evidence.

The course slides describe the procedure as:
1. Start with sense-labelled data such as SemCor.
2. Collect sentences containing the relevant word with a particular sense.
3. Add those contextual sentences to the sense's gloss and examples.
4. Treat the result as the sense signature.
5. Choose the sense whose expanded signature has the greatest overlap with the current context.

This addresses one of the major weaknesses of dictionary-only Lesk: dictionary glosses are often short.

The slides also describe an IDF-weighted version. Instead of simply counting each overlap as one point, each overlapping word receives inverse-document-frequency weight:

```
idf(w) = log(N / df(w))

score(s, context) = Σ idf(w)
```

for overlapping words w.

Common words receive smaller weights; rarer words receive larger weights.

14. Adapted / Extended Lesk

Later work adapts Lesk to WordNet's semantic network. Instead of using only the two original glosses, extended approaches can incorporate glosses of related synsets such as hypernyms, hyponyms, meronyms, holonyms, and other WordNet relations.

The motivation is important: ordinary dictionary glosses are short, so exact lexical overlap may be too sparse. Expanding the semantic neighbourhood provides additional vocabulary with which a target sense can match its context.

Banerjee and Pedersen's extended gloss-overlap work also introduced phrase-sensitive scoring. In that work, an n-word phrase overlap is given an n^2-style score, so a meaningful multi-word overlap can contribute more than n unrelated single-word overlaps.

For teaching purposes, keep this distinction clear:
• Basic/Simplified Lesk → direct word overlap.
• Corpus Lesk → gloss + labelled corpus context, optionally IDF weighting.
• Adapted/Extended Lesk → WordNet relations and expanded glosses; may use richer scoring.

These are extensions, not the same algorithm.

15. Why Lesk works

The algorithm exploits a simple lexical-semantic assumption: nearby words in a coherent sentence tend to be related to the intended sense of the target.

For example:
• “bank + deposit + mortgage” strongly suggests a financial sense.
• “bank + river + canoe” strongly suggests a river-side sense.
• “bass + music + singer” suggests a musical sense.
• “bass + grilled + fish” suggests an animal/food sense.

The algorithm converts this intuition into a measurable score using dictionary evidence.

16. Advantages

1. Simple conceptual model: students can calculate it by hand.
2. No large sense-labelled training corpus is required for basic Lesk.
3. Uses interpretable evidence: the overlapping words can be shown explicitly.
4. Easy to implement.
5. Works with a dictionary or lexical database.
6. Can be adapted to richer resources such as WordNet.
7. It is useful as a knowledge-based baseline for WSD.

The uploaded notes emphasize simplicity and general applicability. However, claims of high accuracy should be treated cautiously; performance depends heavily on the lexical resource, preprocessing, context, and variant of Lesk used.

17. Limitations — explain these carefully

1. Sparse overlap:
Dictionary glosses are short. Two semantically related senses may share no exact words.

2. Exact lexical matching:
Basic Lesk may fail when related words are expressed using synonyms or morphological variants that are not normalized.

3. Context dependence:
A very small window may omit useful evidence; a very large window may add irrelevant words.

4. Dictionary dependence:
Different dictionaries divide meanings differently and provide different glosses/examples.

5. Polysemy with closely related senses:
If two candidate senses are semantically close, their glosses may not provide a decisive lexical distinction.

6. Ties:
Two senses can receive the same overlap score.

7. Part-of-speech ambiguity:
The system must know or correctly handle the relevant POS when retrieving candidate senses.

8. Sense-inventory mismatch:
A real-world usage may not map neatly to one discrete dictionary sense.

9. Preprocessing sensitivity:
Stemming, lemmatization, stop-word removal, phrase matching, and case handling can change the result.

10. Knowledge acquisition / resource coverage:
Words absent from the dictionary or with poor glosses cannot be handled well by a dictionary-only implementation.

A particularly important correction to some simplified teaching material: saying that Lesk “does not consider the context of the word in the sentence” is misleading. Context is central to Lesk; the limitation is that basic implementations represent context through a relatively simple lexical overlap signal.

18. Computational complexity

Let:
k = number of candidate senses of the target word
g = average number of words in a sense signature
c = number of words in the processed context

With efficient set representations, overlap checking can be approximately $O(k\cdot(g+c))$ after preprocessing, or $O(k\cdot g)$ when context membership is implemented as a hash set.

The main practical cost is obtaining and preparing the candidate sense signatures, especially in extended variants that traverse WordNet relations.

19. Original Lesk vs Simplified Lesk vs Corpus/Extended Lesk

Original Lesk:
• Uses dictionary definitions of the target and surrounding words/senses.
• More indirect.
• Classic example: pine cone.
• Closely reflects Lesk's 1986 formulation.

Simplified Lesk:
• Compares the target sense signature directly with the current context.
• Easier to compute and teach.
• Common classroom/exam form.
• Typical score = number of overlapping content words.

Corpus Lesk:
• Adds sense-labelled corpus sentences to the signature.
• Can use IDF weighting.
• Combines dictionary and corpus evidence.

Adapted/Extended Lesk:
• Uses richer semantic information, especially WordNet relations.
• Expands gloss information.
• Can use more sophisticated overlap scoring, including phrase-sensitive scoring.

If an exam simply says “Apply Lesk’s algorithm” and gives a target word, candidate definitions, and a sentence, the safest procedure is to first identify whether the question expects the original or simplified version. If it gives direct gloss-versus-context scoring, it is the simplified version.

20. Common student mistakes

Mistake 1: Choosing the meaning that “sounds right” without calculating overlaps.
Fix: show every candidate sense and its score.

Mistake 2: Counting the target word itself as evidence.
Fix: exclude the target from the context when doing the manual calculation unless the question explicitly says otherwise.

Mistake 3: Counting stop words.
Fix: remove words such as the, a, an, and, of, to, in when the problem uses a stop list.

Mistake 4: Treating synonyms as automatic matches.
Fix: basic Lesk needs lexical overlap. “car” and “automobile” are not the same token unless the resource/variant explicitly handles that relationship.

Mistake 5: Matching “deposit” with “deposits” without saying why.
Fix: either use exact matching or explicitly state that lemmatization is being used.

Mistake 6: Confusing original Lesk with simplified Lesk.
Fix: original Lesk compares sense signatures of target and context words; simplified Lesk directly compares the target sense signature with the context.

Mistake 7: Ignoring ties.
Fix: state the tie and apply a documented tie-breaker.

Mistake 8: Assuming WordNet sense number 1 is always correct.
Fix: sense numbers reflect frequency ordering in WordNet but are not a substitute for the Lesk calculation.

21. How to teach the algorithm in 20–30 minutes

Part 1 — 3 minutes:
Start with one ambiguity:
“The bank is near the river” vs “I deposited money in the bank.”

Ask: same spelling, different meaning — what evidence tells us which meaning is intended?

Part 2 — 5 minutes:
Introduce four terms only:
target word, sense, gloss, context.

Part 3 — 5 minutes:
Write the simplified Lesk formula:

```
Score(sense) = number of overlapping useful words between gloss and context.
```

Then show:
financial bank → deposit, money, loan
river bank → river, water, shore

Part 4 — 7 minutes:
Do the basic “bank” sum with the students. Make them write the score for every sense.

Part 5 — 7 minutes:
Do the advanced “plant” sum and introduce lemmatization.

Part 6 — 3 minutes:
Only after the core idea is understood, explain:
original Lesk → Corpus Lesk → IDF → extended/adapted Lesk.

The key teaching principle is to avoid introducing IDF or WordNet graph relations before students can manually calculate a simple overlap score.

22. Exam-ready answer format

If asked “Explain Lesk’s algorithm”, use this structure:

1. Definition:
Lesk is a knowledge-based/dictionary-based WSD algorithm proposed by Michael Lesk in 1986.

2. Principle:
The correct sense is expected to have the greatest semantic/lexical overlap with the surrounding context, represented through dictionary definitions.

3. Inputs:
Target word, candidate senses, context, and a dictionary/lexical resource.

4. Procedure:
Preprocess context → obtain sense glosses/signatures → compute overlap for every sense → choose maximum-score sense.

```
5. Formula:
Score(si, C) = |Signature(si) ∩ C|
```

6. Result:
The sense with maximum overlap is selected.

7. Example:
Use bank + deposit/money versus bank + river/water.

8. Limitation:
Glosses are short and exact lexical overlap can be sparse; performance depends on dictionary, context, and preprocessing.

If asked for a numerical problem, always show:
Context → Signature 1 → overlap/score → Signature 2 → overlap/score → winner.

23. Quick-reference cheat sheet

LESK = overlap-based knowledge-driven WSD.

Core idea:
Context words + dictionary glosses → overlap score → highest score wins.

```
Core formula:
Score(s, C) = |Signature(s) ∩ C|
```

Basic steps:
1. Identify target word.
2. List candidate senses.
3. Build context.
4. Preprocess.
5. Build each sense signature.
6. Count overlaps.
7. Compare scores.
8. Select maximum.
9. Handle ties explicitly.

Original Lesk:
Target sense signature ↔ signatures of surrounding words.

Simplified Lesk:
Target sense signature ↔ current context.

Corpus Lesk:
Gloss/examples + labelled corpus sentences.

IDF Lesk:
Overlap words receive IDF weights.

Extended/Adapted Lesk:
Use related WordNet synsets and richer overlap scoring.

Main weakness:
Short glosses + exact lexical matching can produce sparse or misleading overlap.

Main strength:
Simple, interpretable, knowledge-based, and easy to calculate manually.

24. Source alignment and references

Uploaded course resources used:
• WSD notes.pdf — contains the simplified Lesk algorithm, pseudocode, the bank example, the original Lesk pine/cone

example, and Corpus Lesk/IDF material.
• lesk algorithm.docx — introduces Lesk as a knowledge-based WSD method and summarizes its steps and limitations.
• Module 5 lecture slides.pdf — places Lesk under knowledge-based WSD and covers WordNet, WSD, simplified Lesk, Corpus Lesk, IDF weighting, and WSD evaluation.
• WSD.docx — gives the course framing of WSD, dictionary-based methods, simplified Lesk, and WSD approaches/applications.

External references:
1. Lesk, M. (1986), “Automatic sense disambiguation using machine readable dictionaries: how to tell a pine cone from an ice cream cone,” SIGDOC.
2. Princeton WordNet documentation — synsets, senses, glosses, and semantic relations.
3. NLTK documentation — implementation of classic Lesk and example using “bank”.
4. Banerjee & Pedersen (2002), “An Adapted Lesk Algorithm for Word Sense Disambiguation using WordNet.”
5. Banerjee & Pedersen (2003), “Extended Gloss Overlaps as a Measure of Semantic Relatedness.”

Worked-sum score tables at a glance

Example	Candidate sense	Overlap	Score
Basic: bank	financial bank	money	1
Basic: bank	river bank	none	0
Advanced: plant	living plant	none	0
Advanced: plant	industrial plant	machine, production	2
Advanced: plant	planting action	none	0

One-minute memory trick

GLOSS + CONTEXT → OVERLAP → SCORE → MAXIMUM → SENSE

If students remember only one line, make them remember this. Everything else is an implementation detail or extension.